

ECE8743 Advanced Robotics

Spring 2025

Milestone Project 1

Reduced and Improved Visibility Graph for Robot Path Planning

Student: Meshaal Mouawad

Student NetID: mm4922

Student E-mail: mm4922@msstate.edu

Instructor: Dr. Chaomin Luo

Department of Electrical and Computer Engineering
Bagley College of Engineering
Mississippi State University

Date: February 14, 2025

Table of Contents

I.	Project Objectives	1
II.	Introduction	Error! Bookmark not defined.
III.	Description of the Project	Error! Bookmark not defined.
III.	Explanation.....	Error! Bookmark not defined.
IV.	SIMULATION RESULTS.....	2
	1) Scenario two	Error! Bookmark not defined.
	2) Scenario three.....	Error! Bookmark not defined.
	3) Scenario Four	Error! Bookmark not defined.
V.	CONCLUSION	Error! Bookmark not defined.
VI.	REFERENCES	8
VII.	APPENDIX.....	8

REDUCED AND IMPROVED VISIBILITY GRAPH FOR ROBOT PATH PLANNING

Meshaal. Mouawad

Department of Electrical and Computer Engineering, Mississippi State University

This technical report is to satisfy the requirement of Advanced Robotics Course, 2025.

Technical Report

Mm4922@msstate.edu

I. PROJECT DESCRIPTION

The objective of the project is to apply the Visibility Graph (VG) for robot path planning and attempt to improve it. We implemented Visibility Graph in multiple scenarios environments. The environments have different obstacles numbers and shapes. We analyze the times elapsed and the total distant. We improved it by reducing the number of the edges and recorded the time and the distant as it shows in table 2. We used Reduced Visibility Graph (RVG) such as the Edge Removal Methodology, and some manual removal functions in MATLAB. We Implemented the Tangent Graph method and finally compared all the methods based on the simulation on MATLAB.

II. INTRODUCTION

VISIBILITY graph (VG) is one of the earliest motion planning algorithms and notable as it is a **complete, optimal** algorithm for polygonal obstacles. The VG Algorithm is used to determine which vertices in a graph are "visible" from a specific observation point. visibility graph is used in computer vision, robotics, and graph theories. The visibility graph algorithm can also be extended to find the shortest path to a visible vertex. Visibility graph is a roadmap approach graph of intervisible locations, typically for a set of points and obstacles in the *Euclidean plane*. Each *vertex* in the graph represents a point location, whereas each *edge* represents a *visible connection* between them. That is, if the line segment connecting two vertices (locations) does not pass through any obstacle, an *edge* is created between them in the graph. When the set of locations lies in a line, this can be addressed as an ordered series [1]. The solution path will pass directly through obstacle corners as shown in Fig. 1.

The methodology to construct visibility graph can be summarized in three steps:

1. The *vertices* are the *start, destination*, and all *obstacle vertices*.

2. The *edges* consist of all possible *collision-free line segments* and *obstacle edges*.
3. The solution paths will pass directly through obstacle corners as shown in Fig. 1.

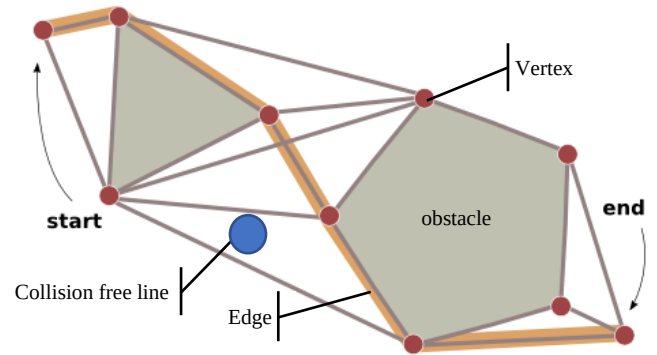


Fig. 1. Demestration of the Visibility Graph.[2]

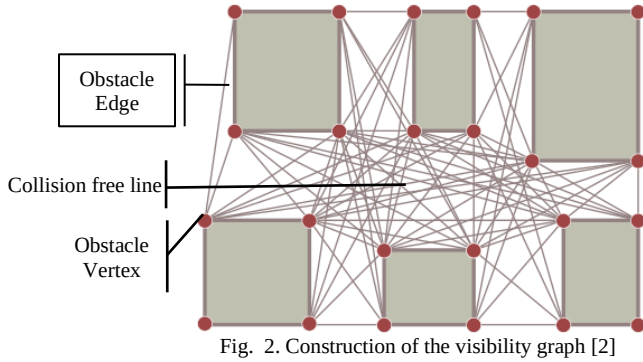
TABLE I
VISIBLE GRAPH ALGORITHM PSEUDOCODE

Visible Graph Algorithm
▪ Let $V \leftarrow \{start, destination\}$ & all obstacle vertices. ▪ $E \leftarrow \{\}$ ▪ For all pairs of distinct vertices $u, v \in V$ ▪ If $\overline{uv}$ is an obstacle edge ▪ Add (u, v) to E. ▪ Else if $\overline{uv}$ is collision free ▪ Add (u, v) to E. ▪ Search $G = (V, E)$, with cartesian distance as the edge cost, to connect start and destination. ▪ Return the path if one is found.

III. VISIBILITY GRAPH ALGORITHMS PROCEDURE

The visibility graph is created by checking for intersections between potential edges (lines connecting vertices) and obstacle edges as explain in section I. Valid edges are added to the graph

G , which represents navigable paths. Then any edges that would cross through the obstacles is removed, ensuring that the graph only contains navigable paths. After the roadmap has been created, then we must find the shortest path from the start to the destination. Table 1. shows the pseudocode of the Visibility graph. The flowchart for the Visibility graph is also presented in figure 3.



Dijkstra's Algorithm is used with VG to determine the shortest path in the map for the robot navigation around the obstacles. The methodology the Dijkstra's Algorithm is maintaining a priority queue of vertices to explore based on their current shortest distance. It iteratively updates the shortest distance to each neighboring vertex until it finds the shortest path to the destination. After the shortest path is found, the total Euclidean distance of the path was calculated which is using a custom function `EuclDist`, summing the distances between consecutive vertices in the path.

IV. SIMULATION RESULTS

We have implemented seven maps for VG with different environments, Environment 1, Environment 2, Environment 3, Environment 4, Environment 5, and improved version of Environment 4 and 5. Each environment has been recorded in Table 2. And discussed its results in this section.

Visibility Graph

We implemented the VG with 9 obstacles as shown in Fig. 1, the Diistant Traveled was 144.8128, and the Elapsed Time: 0.765113 s

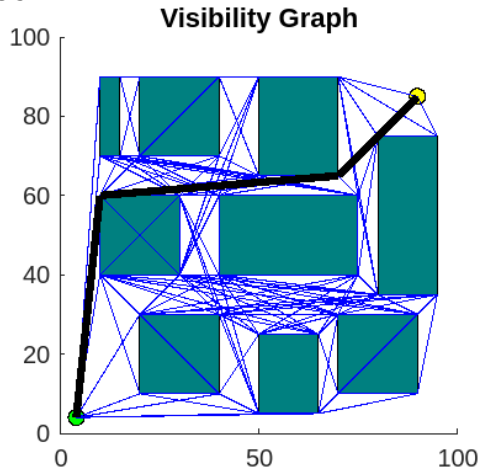


Fig. 1. the visibility graph with 9- obstacles and the generated path planning from the user screen on MATLAB.

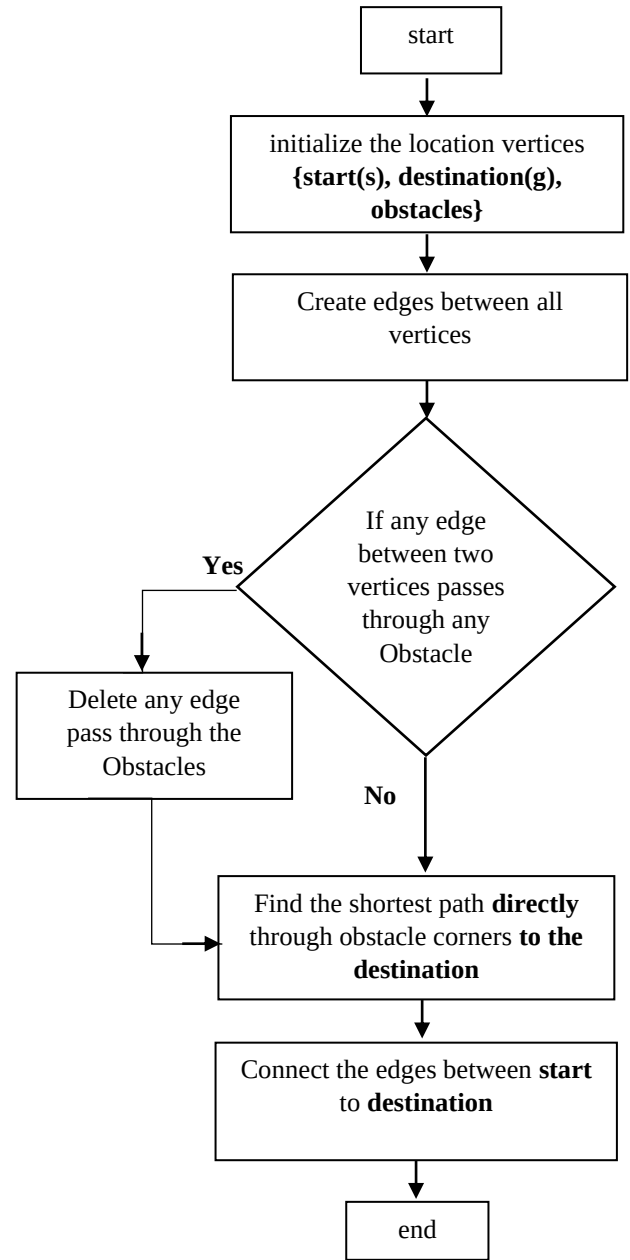


Figure 3. The architecture of the Visibility Graph path planning algorithm.

Visibility Graph for Enviroment 1

We implemented the VG with 2 obstacles as shown in Fig. 2, the Diistant Traveled was 137.0980, and the Elapsed Time: 0.221732 s

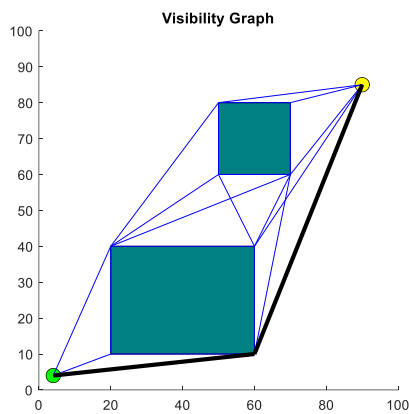


Fig. 2. Environment 1 of the visibility graph with 2- obstacles and the generated path planning from the user screen on MATLAB.

Visibility Graph for Enviroment 2

We implemented the VG with 4 obstacles as shown in Fig. 3, the Diistant Traveled was 137.8629, and the Elapsed Time: 0.448631 s.

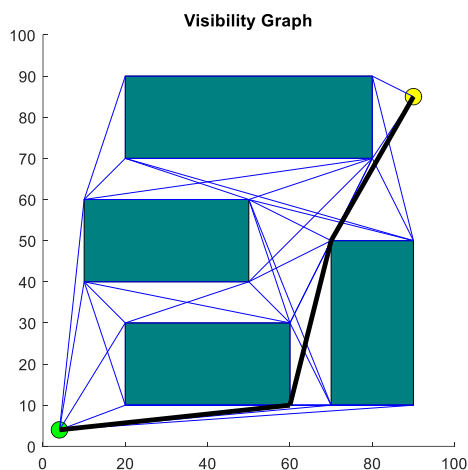


Fig. 3. Environment 2 of the visibility graph with 4- obstacles and the generated path planning from the user screen on MATLAB.

Visibility Graph for Enviroment 3

We implemented the VG with 4 obstacles as shown in Fig. 4, the Diistant Traveled was 144.8128, and the Elapsed Time: 0.694356 s

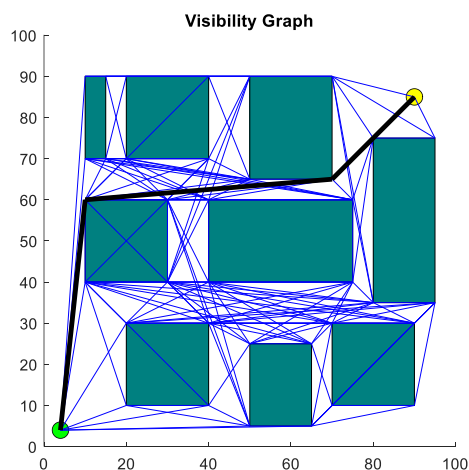


Fig. 4. Environment 3 of the visibility graph with 9- obstacles and the generated path planning from the user screen on MATLAB.

Visibility Graph for Enviroment 4

Fig. 8, is Map (1) generated with the visible graph for Enviroment 4 with the Initial point (0.5,15.5), and the goal (15.5,0.5). this is the initial figure without reduced and improved the visibility. The total distant 34.42 and the elepsed time 0.344 s as recorded in Table 2.

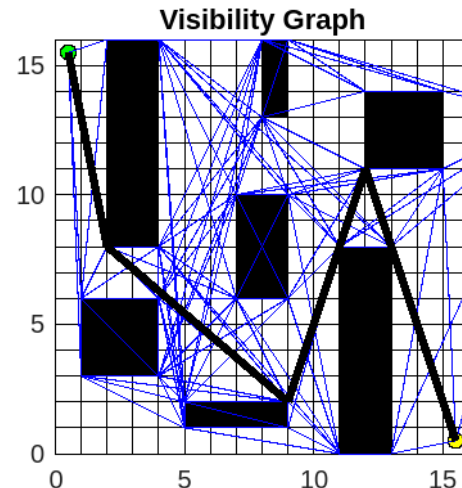


Fig. 8. Environment 4 of the visibility graph for Map-1 with the obstacles and the generated path planning from the user screen on MATLAB.

Visibility Graph for Enviroment 5

We implemented the VG with 15 obstacles as shown in Fig. 9, the Diistant Traveled was 2.4649e+03, and the Elapsed Time: 0.830651 s. We estimated and measured the *starting*, *target points*, and the *obstacle positions* for Map (2). In Fig. 8. Based on the MATLAB and the simulation results:

a. The Starting Position:

- Coordinates: (200, 200)

b. The Target Position:

- Coordinates: (1800, 1800)

c. The Obstacle Positions

There are 15 obstacles in Map (2) which are defined as polygons using arrays of coordinates. The positions for the obstacles are:

1. Obstacle 1:

- Coordinates: [(410, 0), (370, 200), (500, 390), (890, 0)]

2. Obstacle 2:

- Coordinates: [(210, 410), (0, 600), (200, 700), (370, 600)]

3. Obstacle 3:

- Coordinates: [(190, 820), (210, 1100), (300, 1160), (300, 760)]

4. Obstacle 4:

- Coordinates: [(210, 1250), (160, 1450), (270, 1790), (400, 1500), (300, 1200)]

5. Obstacle 5:

- Coordinates: [(640, 390), (550, 500), (550, 610), (700, 610), (810, 500)]
6. Obstacle 6:
- Coordinates: [(450, 850), (500, 1100), (650, 1200), (630, 790)]
7. Obstacle 7:
- Coordinates: [(500, 1800), (600, 1950), (780, 1900), (880, 1620)]
8. Obstacle 8:
- Coordinates: [(900, 300), (850, 600), (1100, 650), (1220, 350)]
9. Obstacle 9:
- Coordinates: [(900, 950), (810, 1250), (920, 1270), (1100, 1190), (1100, 880)]
10. Obstacle 10:
- Coordinates: [(1300, 380), (1250, 650), (1300, 780), (1400, 750), (1430, 500)]
11. Obstacle 11:
- Coordinates: [(1210, 900), (1180, 1200), (1300, 1250), (1400, 1050)]
12. Obstacle 12:
- Coordinates: [(1180, 1550), (1100, 1800), (1180, 1850), (1390, 1800), (1320, 1500)]
13. Obstacle 13:
- Coordinates: [(1610, 530), (1500, 760), (1600, 800), (1750, 600)]
14. Obstacle 14:
- Coordinates: [(1500, 1350), (1450, 1620), (1550, 1750), (1670, 1450)]
15. Obstacle 15:
- Coordinates: [(1650, 1090), (1750, 1300), (1990, 1200), (1900, 1000)]

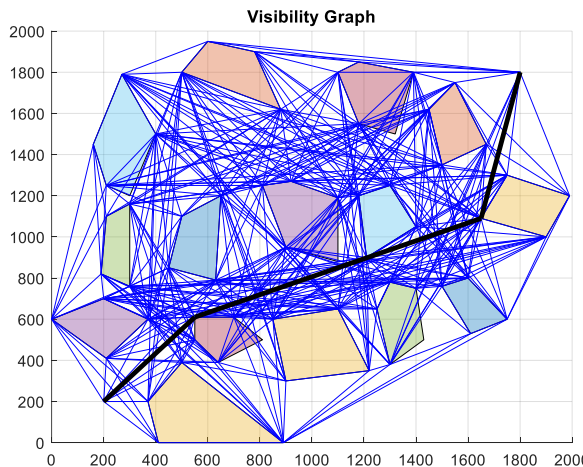


Fig. 9. Environment 5 of the visibility graph for Map (2) and the generated path planning from the user screen on MATLAB.

Visibility Graph for Enviroment 6

We defined the Starting Point (Red Circle) at (10, 90). Target Point (Yellow Circle) at (78, 25).

Shortest path estimation, as it shows in the black line in Fig. 10. Then, we used the Euclidean distance equation. The shortest path coordinates:

[(10,90),(33,70),(48,66),(56,48), (71,43), (75,39), (80, 35), (77, 30)]

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (\text{Eq. 1})$$

The total distant:
99.4037

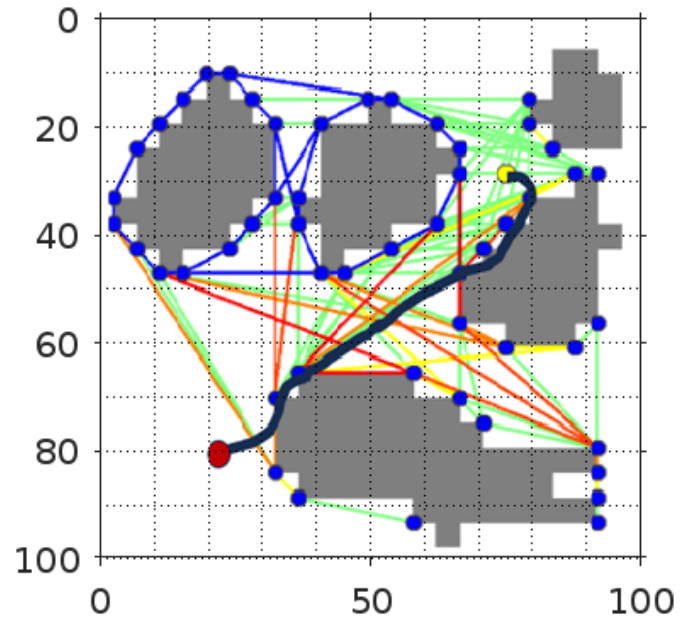


Fig. 10. Environment 6 of the visibility graph for Map-3 with the obstacles and the generated path planning from the user screen on MATLAB.

V. IMPROVEMENT OF VISIBILITY GRAPH

In this project we implemented multiple methods for improving the VG and reduced the edges. We used RVG which is the VG without non-tangent segments and concave vertices. We reduced the visibility graph by removing the edges that are tangent that do not head into the object at either point. We checked the tangential condition and then removed the edges accordingly. The methodology is, checking tangential Edge by Adding a function `isTangential` to check if the edge connects tangentially to the obstacles. Finally, we Checked the Intersection with a function `isIntersecting` to determine if two line segments intersect, allowing for a more robust removing of edges.

We implemented this RVG on Environment 2 as shown in Fig. 14. The elapsed time was 0.145533 s, and the total distant was 161.2090 which is increased in the distant but decreased in the elapsed time. This can be explained due to the number of edges

generated, the elapsed time decreased and due to the reduced of the edges the optimal path planning is limited; therefore, the distance was increased. The removing process of reducing the edges can be overly restrictive if not handled carefully. If the intersection checks are too strict, they may prevent valid paths from being added to the graph. The method ensures that only valid connections (those that don't intersect obstacles) are included in the graph.

To explain the complexity, it was shown in maps with lots of open areas or long corridors such as Fig. 15, visibility graphs complexity increased since every pair of obstacle vertices are connected. These connections between the vertices such that N corners get to N^2 edges [2]. Using some algorithms may remove the redundant edges. However, removing is also increased the complexity in terms of the large number of edges [2]. A proposed optimization is to only look at nearby nodes. Also we may remove the edges that aren't tangent to both vertices (these will never be in the shortest path) [2].

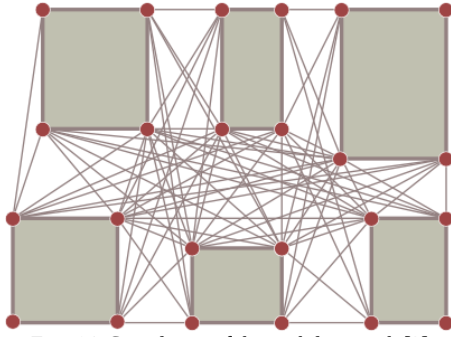


Fig. 14. Complexity of the visibility graph [2]

TABLE 3
REDUCED VISIBLE GRAPH ALGORITHM PSEUDOCODE

Reduced Visible Graph Algorithm

1. **Initialize** nodes from **start**, **goal**, and **obstacle vertices**.
2. **Initialize** an empty graph **G**.
3. **For** each pair of nodes (**p1**, **p2**):
 - a. **If** **p1** == **p2**, **continue** to the next pair.
 - b. **Initialize** a flag variable `isValidEdge` to true.
 - c. **For** each edge (obstacle edge):
 - i. Check **if** the line segment connecting **p1** and **p2** intersects with the obstacle edge.
 - ii. **If** it intersects:
 - **Set** `isValidEdge` to false and **break** the loop.
 - d. **If** `isValidEdge` is true:
 - i. Check **if** the edge is tangent to obstacles:
 - **If** it is tangent, **add** the edge (**p1**, **p2**) to graph **G**.
5. **Return** the graph **G** with the reduced edges.

The Pseudocode is used in MATLAB

Proposal for Improvement of the RVG shortest path planning

We previously discussed the distant increase after the RVG presented in Environment 2 in Fig. 13. For this reason, the shortest path is not optimal, so we proposed the following improvements that may reduce the total distant:

- a. **Refine Intersection Logic:** Ensure that the intersection checks do not exclude valid edges. This can involve revisiting how edges are filtered.
- b. **Add More Nodes:** adding more intermediate nodes or waypoints to create more potential connections, which can lead to more optimal paths.
- c. **Path Smoothing:** After finding a path, apply techniques to smooth the path, reducing unnecessary turns or segments.
- d. **Use Alternative Algorithms:** employing other pathfinding algorithms, such as A* in conjunction with visibility graph to refine the pathfinding process.

Tangent Graph-based path planning

To modify the provided Visibility Graph (VG) for Tangent Graph-based path planning, we implemented a method that connects points tangentially around obstacles, creating a more efficient pathfinding solution as it shows in Fig. 13. This involves creating tangent lines between obstacle edges and connecting these tangents to form the graph for navigation.

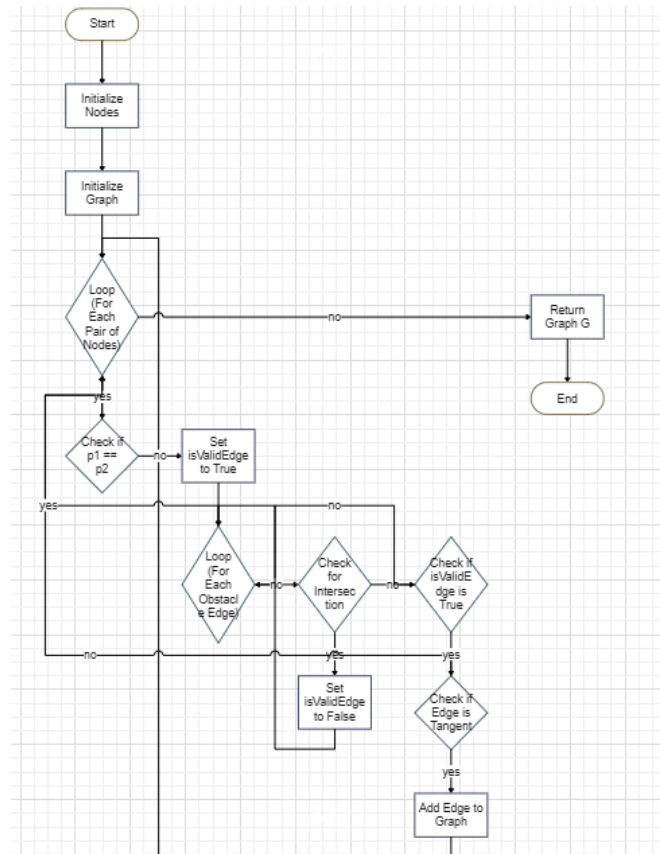


Fig. 15. RVG flowchart after the RVG implemented on Environment 2 that resulted from Fig. 13.

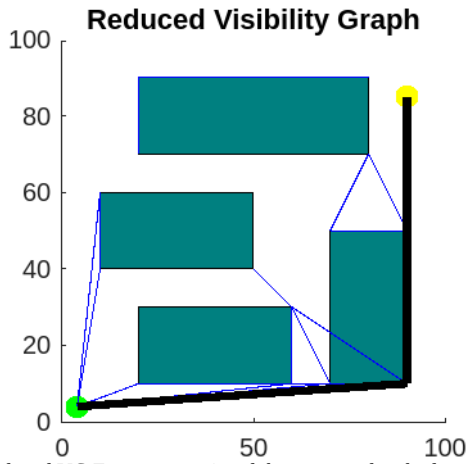


Fig. 14. Reduced VG Environment 2 and the generated path planning from the user screen on MATLAB.

Tangent Graph Methodology:

1. Identifying Tangent Points: For each pair of obstacles, compute tangent points where a straight line can touch the obstacles without intersecting them.
2. Creating Tangent Edges: Connect these tangent points to form edges in the graph.
3. Finding the Shortest Path: Utilize Dijkstra's algorithm or a similar method to find the shortest path from the start to the goal. [13]

TABLE 4

TANGENT GRAPH-BASED PATH PLANNING PSEUDOCODE

Tangent Graph-based path planning

1. **Initialize Start and Goal Positions**
2. **Define Obstacles and Plot Them**
3. **Extract Tangent Points for Each Pair of Obstacles**
4. **Create Tangent Edges Between Tangent Points**
5. **Add Start and Goal Points to the Graph**
6. **Build Graph with Tangent Edges**
7. **Find Shortest Path in the Tangent Graph**
8. **Plot Shortest Path**
9. **Display Total Distance**

The Pseudocode is used in MATLAB

Reduced Visibility Graph & The Tangent Graph

The edge reduction method used in the Environment 5, Map (2) through intersection checks between potential edges and obstacle edges, along with explicit removal of certain edges that are known to be invalid. This ensures that the visibility graph accurately represents valid paths without collisions with obstacles. As it shows in Fig. 12. The shortest path planning is not optimal, it increased to $3.2474e+03$ and the elapsed time decreased to 0.521 seconds. Table. 5 shows the pseudocode used in RVG.

Edge Removal Methodology

Initial Edge Construction:

We constructed an initial set of edges based on the coordinates of the vertices of the obstacles. This is done in the loop where the edges are created using pairs of points.

Checking for Intersections:

For each edge being considered for addition to the graph, if it intersects with any of the edges that represent the boundaries of the obstacles. This is done using the slope and intercept calculations for each edge.

Removing Diagonal Edges:

We implemented a segment that removes specific edges based on their index. It removes edges that correspond to the diagonal connections across the obstacles. This is handled in the following loop:

```
while i < (length(2)-2)
    G = rmedge(G, keys{i}, keys{i+2});
    G = rmedge(G, keys{i+1}, keys{i+3});
    i = i + 4;
end
```

This effectively removes edges that connect vertices that are not valid due to being obstructed by obstacles.

Manual Edge Removal:

Additionally, we manually calls to rmedge to remove specific edges that are identified as not valid connections due to proximity to obstacles:

```
G = rmedge(G, keys{12}, keys{14});
G = rmedge(G, keys{11}, keys{16});
```

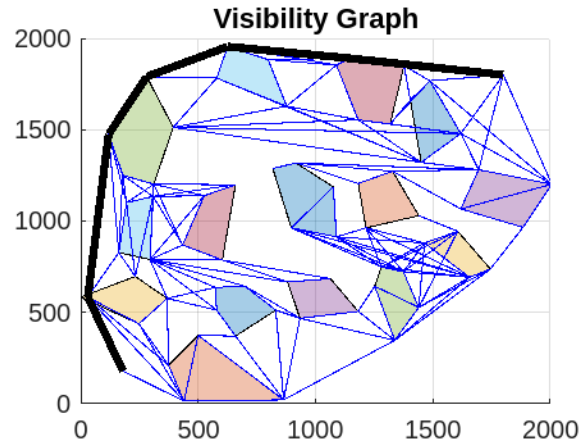


Fig. 13. Map (2) reduced visibility graph of the visibility graph and the generated path planning from the user screen on MATLAB.

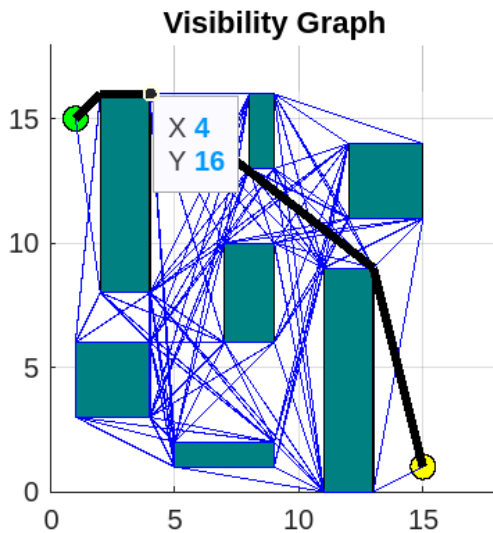


Fig. 12. Improved Environment 4 of the visibility graph for Map (1) and the generated path planning from the user screen on MATLAB.

VI. TG MODEL V.S VG, AND RVG MODELS

We compared the Tangent Graph, the Visibility Graph and Reduced Visibility Graph in terms of the time elapsed (construction time) and the distant travel (pathfinding time).

1. Elapsed Time

Visibility Graph:

Construction Time: Typically longer due to a larger number of edges connecting all visible points.

Pathfinding Time: Can be slower due to the higher complexity of the graph with many edges.

Reduced Visibility Graph:

Construction Time: Faster than the fully visibility graph since unnecessary edges are removed.

Pathfinding Time: Generally improved due to reduced complexity, allowing for quicker computations.

Tangent Graph:

Construction Time: May vary depending on the complexity of the tangent calculations, but it often remains efficient.

Pathfinding Time: Typically faster than both visibility and reduced visibility graphs due to fewer edges and optimized connections.

2. Distance Traveled

Visibility Graph:

Distance: May not always yield the shortest path because it includes many direct connections that could lead to longer routes due to detours around obstacles.

Reduced Visibility Graph:

Distance: Generally shorter than the full visibility graph since only valid edges are retained, although it may not always be optimal.

Tangent Graph:

TABLE 5

MAP (2) REDUCED VISIBILITY GRAPH PSEUDOCODE

Map (2) RVG Pseudocode			
1.	Initialize the environment variables (<i>start, goal, obstacles</i>)		
2.	Create a figure for plotting		
3.	Define obstacle shapes and their coordinates		
4.	Initialize keys and values for nodes (<i>start, goal, obstacle vertices</i>)		
5.	Construct edges based on obstacle vertices:		
	a. For each obstacle vertex pair (<i>i, i+1</i>):		
	i. Create an edge from values[<i>i</i>] to values[<i>i+1</i>]		
	ii. Store the edge in the edges list		
6.	Initialize a graph <i>G</i>		
7.	For each pair of nodes (<i>i, j</i>):		
	a. Calculate line equation for the edge from values[<i>i</i>] to values[<i>j</i>]		
	b. Set flag = 1 (valid edge)		
	c. For each edge in edges:		
	i. Check for intersection with the current edge		
	ii. If an intersection is found:		
	- Set flag = 0 (invalid edge)		
	- Break the loop		
	d. If flag == 1:		
	- Add edge (<i>i, j</i>) to graph <i>G</i>		
8.	Remove invalid edges (diagonal connections):		
	a. For each key index <i>i</i> :		
	i. Remove edges between keys[<i>i</i>] and keys[<i>i+2</i>]		
	ii. Remove edges between keys[<i>i+1</i>] and keys[<i>i+3</i>]		
9.	Manually remove specific edges identified as invalid		
10.	Plot all visible edges in graph <i>G</i>		
11.	Find the shortest path from <i>start</i> to <i>goal</i> in graph <i>G</i>		
12.	Calculate the total distance of the path		

The Pseudocode is used in MATLAB

Distance: Often results in the shortest path since it connects to tangent points on obstacles, minimizing detours and ensuring a more direct route.

Summary of Comparison

Model	Elapsed Time (Construction)	Elapsed Time (Pathfinding)	Distance Traveled
Visibility Graph	Long	Longer	Variable (not optimal)
Reduced Visibility Graph	Moderate	Moderate	Shorter than visibility
Tangent Graph	Efficient	Fast	Often shortest

In general, the Tangent Graph model is likely to outperform both the Visibility Graph and Reduced Visibility Graph models in terms of both elapsed time and distance traveled, especially in environments with complex obstacles. The Reduced Visibility Graph improves on the Visibility Graph by eliminating unnecessary edges, while the Tangent Graph optimizes the pathfinding process further by focusing on tangent connections. The visibility graph is complete but not impractical.

VII. REFERENCES

- [1] [de Berg et al. \(2000\)](#), sections 5.1 and 5.3; [Lozano-Pérez & Wesley \(1979\)](#).
- [2] Lozano-Pérez, Tomás; Wesley, Michael A. (1979), "An algorithm for planning collision-free paths among polyhedral obstacles", *Communications of the ACM*, 22 (10): 560-570, [doi:10.1145/359156.359164](#), [S2CID 17397594](#)
- [3] Fig.1 [Map representations \(stanford.edu\)](#)
- [4] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Communications of the ACM*, vol. 22, no. 10, pp. 560-570, 1979, [doi: 10.1145/359156.359164](#).
- [5] A. Patel, "Amit's Thoughts on Pathfinding.," ed. Stanford University, 1997, p. Demonstration of the Visibility Graph
- [6] M. H. Lee, *Intelligent robotics* (Open University Press robotics series). New York
Milton Keynes: Halsted Press ;
Open University Press, 1989, pp. xiv, 210 p.
- [7] J. F. Engelberger, *Robotics in service*, 1st MIT Press ed. Cambridge, Mass.: MIT Press, 1989, pp. 248 p., 12 p. of plates.
- [8] K. Fujimura, *Motion planning in dynamic environments* (Computer science workbench). Tokyo ; New York: Springer-Verlag, 1991, pp. xiii, 178 p.
- [9] V. D. Hunt, *Understanding robotics*. San Diego: Academic Press, 1990, pp. xvii, 286 p.
- [10] J.-C. Latombe, *Robot motion planning* (The Kluwer international series in engineering and computer science, no. SECS 124). Boston: Kluwer Academic Publishers, 1991, pp. xviii, 651 p.
- [11] D. A. Kelly, *A layman's introduction to robotics*. Princeton, N.J.: Petrocelli Books, 1986, pp. xxvii, 209 p.
- [12] J. P. Laumond, *Robot motion planning and control* (Lecture notes in control and information sciences, no. 229). London ; New York: Springer, 1998, pp. xii, 343 p.
- [13] "<Yun-hui Liu Tangent Graph PP Polygonal and Curved Obstacles IJRR1992.pdf>."

VIII. APPENDIX

Here is the code for the simulation which was completed on MATLAB.

ECE8743_Visible_graph.m

```
clc
close all
clear
tic
%% Defining environment variables
start = [4,4]; % start position
goal = [90, 85]; % goal position
n = 2; % no. of obstacles

%% Defining the grid
figure

%% 3rd Config
rectangle('Position',[20 10 20 20], 'FaceColor',[0 .5 .5])% creates a rectangle [x y w h]
axis([0 100 0 100])
axis square
rectangle('Position',[70 10 20 20], 'FaceColor',[0 .5 .5])
rectangle('Position',[10 40 20 20], 'FaceColor',[0 .5 .5])
rectangle('Position',[20 70 20 20], 'FaceColor',[0 .5 .5])
rectangle('Position',[50 5 15 20], 'FaceColor',[0 .5 .5])
```

TABLE 2
RECORDS OF THE ELAPSED TIME AND DISTANCE TRAVELLED.

Parameters	Distance Travelled	Elapsed Time
Visible graph	144.8128	0.765113 s
Visible graph Environment 1	137.0980	0.221732 s
Visible graph Environment 2	137.8629	0.448631 s
Visible graph Environment 3	144.8128	0.694356 s
Visible graph Environment 4	37.4229	0.344897 s
Visible graph Environment 5	2.4649e+03	0.830651 s
Improved VG environments 4	23.06220	0.364330 s
Map (2) Reduced VG	3.2474e+03	0.521206 s
Reduced VG Environment 2	161.2090	0.166360 s

S is the time in seconds. The time complexity is the Elapsed Time which MATLAB® reads the internal time at the execution of the toc function and displays the elapsed time since the most recent call to the tic function without an output.

```
rectangle('Position',[40 40 35 20], 'FaceColor',[0 .5 .5])
rectangle('Position',[10 70 5 20], 'FaceColor',[0 .5 .5])
rectangle('Position',[50 65 20 25], 'FaceColor',[0 .5 .5])
rectangle('Position',[80 35 15 40], 'FaceColor',[0 .5 .5])

% Plotting start position
circles(start(1), start(2),2, 'facecolor','green')

% Plotting goal position
circles(goal(1), goal(2),2, 'facecolor','yellow')

%% initialising the hash map

%% 3rd Config
keys = {'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z','aa','ab','ac','ad','ae','af','ag','ah','ai','aj','ak','al'};
values = {start, [20,10], [20,30], [40,30], [40,10], [50,5], [50,25], [65,25], [65,5], [70,10], [70,30], [90,30], [90,10], [10,40], [10,60], [30,60], [30,40], [40,40], [40,60], [75,60], [75,40], [80,35], [80,75], [95,75], [95,35], [10,70], [10,90], [15,90], [15,70], [20,70], [20,90], [40,90], [40,70], [50,65], [50,90], [70,90], [70,65], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Map = containers.Map(keys, values);
len = Map.values;
Map('j');

%% Building all the obstacle edges
length = size(keys); % this contains the number of nodes

edges = [];

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1), values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
```

```

while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
    values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%%

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);

            if m1==m2 %% ignoring
                t = 1;
            else

                %%%%%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

                temp3 = p1(2) - m2*p1(1) - c2;
                temp4 = q1(2) - m2*q1(1) - c2;

                if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
                    flag = 0;
                    break
                end
                %%%%%%
            end
        end

        end
        if flag==1
            G = addedge(G,keys{i}, keys{j});
        end
    end
end
end

```

```

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;

while i < (length(2)-2)

    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

% G = rmedge(G,keys{15}, keys{32});
% G = rmedge(G,keys{14}, keys{33});
% G = rmedge(G,keys{15}, keys{4});
% G = rmedge(G,keys{14}, keys{5});
% G = rmedge(G,keys{10}, keys{25});
% G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0; % this will contain the total
distance in units for the final selected path

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis
%%%%%%%%%%%%%% end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

ECE8743_Visible_graph_Environment_1.m

clc
close all
clear
tic
%% Defining environment variables
start = [4,4]; % start position
goal = [90, 85]; % goal position
n = 2; % no. of obstacles

%% Defining the grid

```

```

figure
% Configured in Environment 1
%
rectangle('Position',[20 10 40 30], 'FaceColor',[0 0
0]) % creates a rectangle [x y w h]
axis([0 100 0 100])
axis square
hold on
rectangle('Position',[50 60 20 20], 'FaceColor',[0 0
0])
% Plotting start position
circles(start(1), start(2),2, 'facecolor','green')
% Plotting goal position
circles(goal(1), goal(2),2, 'facecolor','yellow')
%% initialising the hash map
% Configured in Environment 1
keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j'};
values = {start, [20,10], [20,40], [60,40],
[60,10], [50,60], [50,80], [70,80], [70,60], goal};
%% Map = containers.Map(keys, values);
len = Map.values;
Map('j');
%% Building all the obstacle edges
length = size(keys); % this contains the number
of nodes
edges = [];
for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end
% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end
%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);
G = graph();
for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        % flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges
            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;

```

```

end
c2 = ed(3) - m2*ed(1);

if m1==m2 %% ignoring
    t = 1;
else
    % temp1 = ed(3) - m1*ed(1) - c1;
    % temp2 = ed(4) - m1*ed(2) - c1;

    temp3 = p1(2) - m2*p1(1) - c2;
    temp4 = q1(2) - m2*q1(1) - c2;

    if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
        flag = 0;
        break
    end
    % flag = 1;
end

end
if flag==1
    G = addedge(G,keys{i}, keys{j});
end

end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;

while i < (length(2)-2)

    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

% G = rmedge(G,keys{15}, keys{32});
% G = rmedge(G,keys{14}, keys{33});
% G = rmedge(G,keys{15}, keys{4});
% G = rmedge(G,keys{14}, keys{5});
% G = rmedge(G,keys{10}, keys{25});
% G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0; % this will contain the total
distance in units for the final selected path

```

```

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

ECE8743_Visible_graph_Environment_2.m

```

clc
close all
clear
tic
%% Defining environment variables
start = [4,4];      % start position
goal = [90, 85];    % goal position
n = 2;              % no. of obstacles

%% Defining the grid
figure

%% 1st Config
% rectangle('Position',[20 10 40 30], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square
%
% hold on
% rectangle('Position',[50 60 20 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% Environment 2 - 2nd Config
rectangle('Position',[20 10 40 20], 'FaceColor',[0
.5 .5])
axis([0 100 0 100])
axis square

rectangle('Position',[70 10 20 40], 'FaceColor',[0
.5 .5])
rectangle('Position',[10 40 40 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[20 70 60 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 3rd Config
% rectangle('Position',[20 10 20 20], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square

% rectangle('Position',[70 10 20 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[10 40 20 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[20 70 20 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[50 5 15 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[40 40 35 20], 'FaceColor',[0
.5 .5])

```

```

% rectangle('Position',[10 70 5 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[50 65 20 25], 'FaceColor',[0
.5 .5])
% rectangle('Position',[80 35 15 40], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

% Plotting start position
circles(start(1), start(2),2, 'facecolor','green')

```

```

% Plotting goal position
circles(goal(1), goal(2),2, 'facecolor','yellow')

```

```

%% initialising the hash map

```

```

%% 1st Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j'};
% values = {start, [20,10], [20,40], [60,40],
[60,10], [50,60], [50,80], [70,80], [70,60], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%% 2nd Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r'};
% values = {start, [20,10], [20,30], [60,30],
[60,10], [10,40], [10,60], [50,60], [50,40],
[70,10], [70,50], [90,50], [90,10], [20,70],
[20,90], [80,90], [80,70], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

%% 3rd Config
%keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r',
's', 't', 'u', 'v', 'w', 'x', 'y', 'z', 'aa', 'ab',
'ac', 'ad', 'ae', 'af', 'ag', 'ah', 'ai', 'aj',
'ak', 'al'};
%values = {start, [20,10], [20,30], [40,30],
[40,10], [50,5], [50,25], [65,25], [65,5], [70,10],
[70,30], [90,30], [90,10], [10,40], [10,60],
[30,60], [30,40], [40,40], [40,60], [75,60],
[75,40], [80,35], [80,75], [95,75], [95,35],
[10,70], [10,90], [15,90], [15,70], [20,70],
[20,90], [40,90], [40,70], [50,65], [50,90],
[70,90], [70,65], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

```

Map = containers.Map(keys, values);
len = Map.values;
Map('j');

```

```

%% Building all the obstacle edges
length = size(keys);    % this contains the number
of nodes

```

```

edges = [];

```

```

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

```

```

% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

```

```

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
    values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%%

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);

            if m1==m2 %% ignoring
                t = 1;
            else

                %%%%%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

                temp3 = p1(2) - m2*p1(1) - c2;
                temp4 = q1(2) - m2*q1(1) - c2;

                if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
                    flag = 0;
                    break
                end
                %%%%%%
            end
        end

        if flag==1
            G = addedge(G,keys{i}, keys{j});
        end
    end
end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;

```

```

while i < (length(2)-2)
    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

% G = rmedge(G,keys{15}, keys{32});
% G = rmedge(G,keys{14}, keys{33});
% G = rmedge(G,keys{15}, keys{4});
% G = rmedge(G,keys{14}, keys{5});
% G = rmedge(G,keys{10}, keys{25});
% G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0; % this will contain the total
distance in units for the final selected path

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis
%%%%%%%%%%%%%% end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

ECE8743_Visible_graph_Environment_3.m


---


clc
close all
clear
tic
%% Defining environment variables
start = [4,4]; % start position
goal = [90, 85]; % goal position
n = 2; % no. of obstacles

%% Defining the grid
figure

%%% 1st Config %%%%
% rectangle('Position',[20 10 40 30], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square

```



```
%
% hold on
% rectangle('Position',[50 60 20 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%% Environment 2 - 2nd Config
% rectangle('Position',[20 10 40 20], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square

% rectangle('Position',[70 10 20 40], 'FaceColor',[0
.5 .5])
% rectangle('Position',[10 40 40 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[20 70 60 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%% 3rd Config
rectangle('Position',[20 10 20 20], 'FaceColor',[0
.5 .5])
axis([0 100 0 100])
axis square

rectangle('Position',[70 10 20 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[10 40 20 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[20 70 20 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[50 5 15 20], 'FaceColor',[0 .5
.5])
rectangle('Position',[40 40 35 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[10 70 5 20], 'FaceColor',[0 .5
.5])
rectangle('Position',[50 65 20 25], 'FaceColor',[0
.5 .5])
rectangle('Position',[80 35 15 40], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Plotting start position
circles(start(1), start(2),2, 'facecolor','green')

% Plotting goal position
circles(goal(1), goal(2),2, 'facecolor','yellow')

%% initialising the hash map

%%% 1st Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j'};
% values = {start, [20,10], [20,40], [60,40],
[60,10], [50,60], [50,80], [70,80], [70,60], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%% 2nd Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r'};
% values = {start, [20,10], [20,30], [60,30],
[60,10], [10,40], [10,60], [50,60], [50,40],
[70,10], [70,50], [90,50], [90,10], [20,70],
[20,90], [80,90], [80,70], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%% 3rd Config
keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's',
't', 'u', 'v', 'w', 'x', 'y', 'z', 'aa', 'ab', 'ac',
```

```
'ad', 'ae', 'af', 'ag', 'ah', 'ai', 'aj', 'ak',
'al'};
values = {start, [20,10], [20,30], [40,30], [40,10],
[50,5], [50,25], [65,25], [65,5], [70,10], [70,30],
[90,30], [90,10], [10,40], [10,60], [30,60],
[30,40], [40,40], [40,60], [75,60], [75,40],
[80,35], [80,75], [95,75], [95,35], [10,70],
[10,90], [15,90], [15,70], [20,70], [20,90],
[40,90], [40,70], [50,65], [50,90], [70,90],
[70,65], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Map = containers.Map(keys, values);
len = Map.values;
Map('j');

%% Building all the obstacle edges
length = size(keys); % this contains the number
of nodes

edges = [];

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
```

```

        m2 = 1e+10;
    end
    c2 = ed(3) - m2*ed(1);

    if m1==m2 %% ignoring
        t = 1;
    else

        %%%%%
        temp1 = ed(3) - m1*ed(1) - c1;
        temp2 = ed(4) - m1*ed(2) - c1;

        temp3 = p1(2) - m2*p1(1) - c2;
        temp4 = q1(2) - m2*q1(1) - c2;

        if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
            flag = 0;
            break
        end
        %%%%%
    end

    end

    if flag==1
        G = addedge(G,keys{i}, keys{j});
    end

end

end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;

while i < (length(2)-2)

    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

% G = rmedge(G,keys{15}, keys{32});
% G = rmedge(G,keys{14}, keys{33});
% G = rmedge(G,keys{15}, keys{4});
% G = rmedge(G,keys{14}, keys{5});
% G = rmedge(G,keys{10}, keys{25});
% G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0; % this will contain the total
distance in units for the final selected path

```

```

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis
%%%%%%%%%%%% end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

ECE8743_Visible_graph_Environment_4.m

clc
close all
clear
tic
%% Defining environment variables
start = [0.5,15.5]; % star
goal = [15.5, 0.5]; % goal position
n = 2; % no. of obstacles

%% Defining the grid
[X,Y]=meshgrid(0:16);
figure; hold on;
plot(X,Y,'k');
plot(Y,X,'k');axis on

axis([0 16 0 16]);
axis square
Ob2 = [1 3 3 3];
Ob3 = [5 1 4 1];
Ob4 = [7 6 2 4];
Ob5 = [8 13 1 3];
Ob6 = [11 0 2 8];
Ob7 = [12 11 3 3];
rectangle('Position',[2 8 2 8], 'FaceColor',[0 0
0])% creates a rectangle [x y w h]
% axis([0 100 0 100])
rectangle('Position',Ob2, 'FaceColor',[0 0 0])
rectangle('Position',Ob3, 'FaceColor',[0 0 0])
rectangle('Position',Ob4, 'FaceColor',[0 0 0])
rectangle('Position',Ob5, 'FaceColor',[0 0 0])
rectangle('Position',Ob6, 'FaceColor',[0 0 0])
rectangle('Position',Ob7, 'FaceColor',[0 0 0])
% Plotting start position
circles(start(1), start(2),0.3, 'facecolor','green')
% Plotting goal position
circles(goal(1), goal(2),0.3, 'facecolor','yellow')

%% initialising the hash map
cor1 = [2,8];
cor2 = [2,16];
cor3 = [4,16];
cor4 = [4,8];

cor5 = [1,3];
cor6 = [1,6];
cor7 = [4,6];
cor8 = [4,3];

cor9 = [5,1];
cor10 = [5,2];
cor11 = [9,2];
cor12 = [9,1];

cor13 = [7,6];
cor14 = [7,10];

```

```

cor15 = [9,10];
cor16 = [9,6];

cor17 = [8,13];
cor18 = [8,16];
cor19 = [9,16];
cor20 = [19,13];

cor21 = [11,0];
cor22 = [11,8];
cor23 = [13, 8];
cor24 = [13. 0];

cor25 = [12, 11];
cor26 = [12, 14];
cor27 = [15, 14];
cor28 = [15, 11];
%% 3rd Config
keys = {'a','b','c','d','e','f','g','h','i',
        'j','k','l','m','n','o','p','q','r','s',
        't','u','v','w','x','y','z',
        'aa','ab','ac','ad'};
values = {start, cor1, cor2, cor3, cor4, cor5, cor6,
          cor7, cor8, cor9, cor10, cor11, cor12, cor13, cor14,
          cor15, cor16, cor17, cor18, cor19, cor20, cor21,
          cor22, cor23, cor24, cor25, cor26, cor27,
          cor28, goal};
% values = {start, , [20,30], [40,30], [40,10],
[50,5], [50,25], [65,25], [65,5], [70,10], [70,30],
[90,30], [90,10], [10,40], [10,60], [30,60],
[30,40], [40,40], [40,60], [75,60], [75,40],
[80,35], [80,75], [95,75], [95,35], [10,70],
[10,90], [15,90], [15,70], [20,70], [20,90],
[40,90], [40,70], [50,65], [50,90], [70,90],
[70,65], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Map = containers.Map(keys, values);
len = Map.values;
Map('ad');

%% Building all the obstacle edges
length = size(keys); % this contains the number
of nodes
edges = [];
for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
            values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
            values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph

```

```

ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);

            if m1==m2 %% ignoring
                t = 1;
            else

                %%%%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

                temp3 = p1(2) - m2*p1(1) - c2;
                temp4 = q1(2) - m2*q1(1) - c2;

                if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
                    flag = 0;
                    break
                end
                %%%%%
            end

            end

            if flag==1
                G = addedge(G,keys{i}, keys{j});
            end

        end

    end

    end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;

while i < (length(2)-2)

    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

% G = rmedge(G,keys{15}, keys{32});
% G = rmedge(G,keys{14}, keys{33});
% G = rmedge(G,keys{15}, keys{4});
% G = rmedge(G,keys{14}, keys{5});
% G = rmedge(G,keys{10}, keys{25});
% G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

```

```

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x(1,1);
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0; % this will contain the total
distance in units for the final selected path

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%% end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

ECE8743_Visible_graph_Environment_5.m

```

clc
close all
clear
tic
%% Defining environment variables
start = [200,200]; % start position
goal = [1800, 1800]; % goal position
n = 2; % no. of obstacles
%% Defining the grid
figure; hold on;
% Create obstacles
ob1 = [410 0; 370 200; 500 390; 890 0];
ob2 = [210 410; 0 600; 200 700; 370 600];
ob3 = [190 820; 210 1100; 300 1160; 300 760];
ob4 = [210 1250; 160 1450; 270 1790; 400 1500; 300 1200];
ob5 = [640 390; 550 500; 550 610; 700 610; 810 500];
ob6 = [450 850; 500 1100; 650 1200; 630 790];
ob7 = [500 1800; 600 1950; 780 1900; 880 1620];
ob8 = [900 300; 850 600; 1100 650; 1220 350];
ob9 = [900 950; 810 1250; 920 1270; 1100 1190; 1100 880];
ob10 = [1300 380; 1250 650; 1300 780; 1400 750; 1430 500];
ob11 = [1210 900; 1180 1200; 1300 1250; 1400 1050];
ob12 = [1180 1550; 1100 1800; 1180 1850; 1390 1800; 1320 1500];
ob13 = [1610 530; 1500 760; 1600 800; 1750 600];
ob14 = [1500 1350; 1450 1620; 1550 1750; 1670 1450];
ob15 = [1650 1090; 1750 1300; 1990 1200; 1900 1000];
g1 = polyshape(ob1); g2 = polyshape(ob2); g3 = polyshape(ob3);

```

[illegible]

```

len = Map.values;
Map('j');
%% Building all the obstacle edges
lenNode = size(keys); % this contains the number
of nodes
edges = [];
for i = 2:(lenNode(2)-2) % start from the second
point (forst point is start point)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end
% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end
% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < lenNode(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end
%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);
G = graph();
for i = 1:lenNode(2)
    for j = (i + 1):lenNode(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);
            if m1==m2 %% ignoring
                t = 1;
            else

                %%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

                temp3 = p1(2) - m2*p1(1) - c2;
                temp4 = q1(2) - m2*q1(1) - c2;

                if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
                    flag = 0;
                    break
                end
            end
        end
        if flag==1

```

```

        G = addedge(G,keys{i}, keys{j});
    end
end

%% Removing the diagonals of the obstacle from the
visible edges
lenNode = size(keys); % this contains the number
of nodes
i = 2;

while i < (lenNode(2)-2)
    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end

%% Plotting all the visible edges
visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it
path = shortestpath(G, keys{1}, keys{lenNode(2)});
pathSize = size(path);
totalDis = 0; % this will contain the total
distance in units for the final selected path

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    title('Visibility Graph')
end

toc
totalDis;

```

ReducedVGenEnv2.m

```

clc
close all
clear
tic
%% Defining environment variables
start = [4,4]; % start position
goal = [90, 85]; % goal position
n = 2; % no. of obstacles

%% Defining the grid
figure

%% Environment 2 - 2nd Config
rectangle('Position',[20 10 40 20], 'FaceColor',[0
.5 .5])
axis([0 100 0 100])
axis square

```

```

rectangle('Position',[70 10 20 40], 'FaceColor',[0
.5 .5])
rectangle('Position',[10 40 40 20], 'FaceColor',[0
.5 .5])
rectangle('Position',[20 70 60 20], 'FaceColor',[0
.5 .5])

% Plotting start position
hold on;
plot(start(1), start(2), 'go', 'MarkerFaceColor',
'green', 'MarkerSize', 8); % Green circle

% Plotting goal position
plot(goal(1), goal(2), 'yo', 'MarkerFaceColor',
'yellow', 'MarkerSize', 8); % Yellow circle

%% Initialising the hash map
keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r'};
values = {start, [20,10], [20,30], [60,30], [60,10],
[10,40], [10,60], [50,60], [50,40], [70,10],
[70,50], [90,50], [90,10], [20,70], [20,90],
[80,90], [80,70], goal};

Map = containers.Map(keys, values);
len = Map.values;

%% Building all the obstacle edges
length = size(keys); % this contains the number
of nodes
edges = [];

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1, i};
        q1 = values{1, j};

        % Skip identical points
        if p1(1) == q1(1) && p1(2) == q1(2)
            continue;
        end

        % Check for intersection with any obstacles
        flag = 1; % Assume edge is valid unless
proven otherwise
        for k = 1:noEdges
            ed = edges(k,:);
            if isIntersecting(p1, q1, ed(1:2),
ed(3:4))
                flag = 0; % Edge intersects an
obstacle
                break;
            end
        end

        % Add edge if valid
        if flag
            G = addedge(G, keys{i}, keys{j});
        end
    end
end

%% Function to check intersection between two line
segments
function intersects = isIntersecting(p1, q1, p2, q2)
    % Calculate the direction of each point
    d1 = direction(p1, q1, p2);
    d2 = direction(p1, q1, q2);
    d3 = direction(p2, q2, p1);
    d4 = direction(p2, q2, q1);

    % Check if the segments straddle each other
    intersects = (d1 * d2 < 0) && (d3 * d4 < 0);
end

%% Function to calculate direction
function dir = direction(p1, q1, p2)
    dir = (p2(1) - p1(1)) * (q1(2) - p1(2)) - (p2(2)
- p1(2)) * (q1(1) - p1(1));
end

%% Debugging: Print the edges in the graph
disp('Current edges in the graph:');
disp(G.Edges);

%% Debugging: Print the nodes in the graph
disp('Current nodes in the graph:');
disp(G.Nodes.Name);

%% Plotting all the visible edges
visEd = G.Edges;
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% Finding the shortest path in the graph and
printing it
path = shortestpath(G, keys{1}, keys{length(2)});
if isempty(path)
    disp('No path found between start and goal.');
```

```

else
    pathSize = size(path);
    totalDis = 0;

    % Plot the shortest path
    for i=1:pathSize(2)-1
        p1 = Map(path{i});
        p2 = Map(path{i+1});

        totalDis = totalDis + EuclDist(p1,p2);

        xpoints = [p1(1,1), p2(1,1)];
        ypoints = [p1(1,2), p2(1,2)];
        hold on
        plot(xpoints, ypoints, 'k', 'LineWidth', 3)
    end
    % Plot the path in black
end

```

```

        title('Reduced Visibility Graph')
    end
    disp(['Total distance: ', num2str(totalDis)]);
end

toc
totalDis

%% Function to calculate Euclidean distance
function d = EuclDist(p1, p2)
    d = sqrt((p2(1) - p1(1))^2 + (p2(2) - p1(2))^2);
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

```

ECE8743_ImprovedVisible_graph_Environment_4.m

```

clc
close all
clear
tic
%% Defining environment variables
start = [1,15];    % start position
goal = [15, 1];    % goal position
n = 2;             % no. of obstacles

%% Defining the grid
figure

%% 1st Config
% rectangle('Position',[20 10 40 30], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square
%
% hold on
% rectangle('Position',[50 60 20 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% Environment 2 - 2nd Config
% rectangle('Position',[20 10 40 20], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square

% rectangle('Position',[70 10 20 40], 'FaceColor',[0
.5 .5])
% rectangle('Position',[10 40 40 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[20 70 60 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 3rd Config
% rectangle('Position',[20 10 20 20], 'FaceColor',[0
.5 .5])
axis([0 18 0 18])
grid on
axis square

rectangle('Position',[1 3 3 3], 'FaceColor',[0 .5
.5])
rectangle('Position',[2 8 2 8], 'FaceColor',[0 .5
.5])
rectangle('Position',[5 1 4 1], 'FaceColor',[0 .5
.5])
rectangle('Position',[7 6 2 4], 'FaceColor',[0 .5
.5])
rectangle('Position',[8 13 1 3], 'FaceColor',[0 .5
.5])

```

```

rectangle('Position',[11 0 2 9], 'FaceColor',[0 .5
.5])
rectangle('Position',[ 12 11  3 3], 'FaceColor',[0
.5 .5])
% rectangle('Position',[80 35 15 40], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% Plotting start position
circles(start(1), start(2),0.5, 'facecolor','green')

% Plotting goal position
circles(goal(1), goal(2),0.5, 'facecolor','yellow')

%% initialising the hash map

%% 1st Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j'};
% values = {start, [20,10], [20,40], [60,40],
[60,10], [50,60], [50,80], [70,80], [70,60], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 2nd Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r'};
% values = {start, [20,10], [20,30], [60,30],
[60,10], [10,40], [10,60], [50,60], [50,40],
[70,10], [70,50], [90,50], [90,10], [20,70],
[20,90], [80,90], [80,70], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 3rd Config
keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's',
't', 'u', 'v', 'w', 'x', 'y',
'z', 'aa', 'ab', 'ac', 'ad'};
values = {start, [1 ,3] , [1,6], [4, 6], [4 ,3],[2
,8],[2,16],[4 16],[4 ,8], [5 ,1] ,[5 ,2] ,[9 ,2] ,[9
,1],[7 ,6] ,[7 ,10] ,[9 ,10] ,[9 ,6],[8 ,13] ,[8
,16] ,[9 ,16] ,[9 ,13],[11 ,0] ,[11 ,9] ,[13 ,9]
,[13 ,0], [12 ,11] ,[12 ,14] ,[15 ,14] ,[15 ,11],
goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Map = containers.Map(keys, values);
len = Map.values;
Map('j');

%% Building all the obstacle edges
length = size(keys);    % this contains the number
of nodes

edges = [];

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Removing edges which are not obstacle edges
sizeEdges = size(edges);
i = 4;
while i < (sizeEdges(1)-1)
    [edges,ps] = removerows(edges,'ind',i);
    sizeEdges = size(edges);
    i = i + 3;
end

% Adding 1 edge pair in each obstacle which were not
added in the earlier

```

```

% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1));
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%

        flag = 1; % flag to check if the edge has
        any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);

            if m1==m2 %% ignoring
                t = 1;
            else

                %%%%%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

                temp3 = p1(2) - m2*p1(1) - c2;
                temp4 = q1(2) - m2*q1(1) - c2;

                if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
                    flag = 0;
                    break
                end
                %%%%%%
            end
        end

        end
        if flag==1
            G = addedge(G,keys{i}, keys{j});
        end
    end
end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys); % this contains the number
of nodes
i = 2;
pair = [keys; values]; % this is used to find the
corresponding letter with its point and what number
it is on the list
while i < (length(2)-2)

```

```

        G = rmedge(G,keys{i}, keys{i+2});
        G = rmedge(G,keys{i+1}, keys{i+3});
        i = i + 4;
    end

    G = rmedge(G,keys{3}, keys{11});
    G = rmedge(G,keys{11}, keys{16});
    G = rmedge(G,keys{17}, keys{8});
    G = rmedge(G,keys{20}, keys{15});
    G = rmedge(G,keys{2}, keys{13});
    % G = rmedge(G,keys{9}, keys{12});

    %% Plotting all the visible edges

    visEd = G.Edges; % this has the visible edges
    sizeEd = size(G.Edges);

    for i=1:sizeEd(1)
        x = visEd(i,1);
        xx = x{1,1};
        p1 = Map(xx{1,1});
        p2 = Map(xx{1,2});
        xpoints = [p1(1,1), p2(1,1)];
        ypoints = [p1(1,2), p2(1,2)];
        hold on
        plot(xpoints, ypoints, 'b')
    end

    %% finding the shortest path in the graph and
    printing it

    path = shortestpath(G, keys{1}, keys{length(2)});
    pathSize = size(path);

    totalDis = 0; % this will contain the total
    distance in units for the final selected path

    for i=1:pathSize(2)-1
        p1 = Map(path{i});
        p2 = Map(path{i+1});

        totalDis = totalDis + EuclDist(p1,p2);

        xpoints = [p1(1,1), p2(1,1)];
        ypoints = [p1(1,2), p2(1,2)];
        hold on
        plot(xpoints, ypoints, 'k', 'LineWidth', 3)
        title('Visibility Graph')
    end

    pairm = [keys; values];

    toc
    totalDis
    %%%%%%%%%%% end
    %%%%%%%%%%%

    ECE8743_ImprovedVisible_graph_Environment_5.m

    clc
    close all
    clear
    tic
    %% Defining environment variables
    start = [180,180]; % start position
    goal = [1800, 1800]; % goal position
    n = 15; % no. of obstacles

    %% Defining the grid
    figure

    %% 1st Config %%%
    % rectangle('Position',[20 10 40 30], 'FaceColor',[0
    .5 .5])

```



```
% axis([0 100 0 100])
% axis square
%
% hold on
% rectangle('Position',[50 60 20 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% Environment 2 - 2nd Config
% rectangle('Position',[20 10 40 20], 'FaceColor',[0
.5 .5])
% axis([0 100 0 100])
% axis square

% rectangle('Position',[70 10 20 40], 'FaceColor',[0
.5 .5])
% rectangle('Position',[10 40 40 20], 'FaceColor',[0
.5 .5])
% rectangle('Position',[20 70 60 20], 'FaceColor',[0
.5 .5])
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 5th Config
% rectangle('Position',[20 10 20 20], 'FaceColor',[0
.5 .5])
% first polygon
T = ([374 210;500 375;864 22;440 15]);
tgon = polyshape(T);
% plot(tgon)
% hold on

% Second shape
P = [0 584; 232 697;368 577; 248 438];
pgon = polyshape(P);
% plot(pgon)
% plot(tgon)
% hold on

% Third shape
E = [ 573 637;
699 644;
831 513;
659 371;
579 513];
egon = polyshape(E);
% plot(Egon)

D = [ 881 667;
1063 685;
1182 506;
934 468];
dgon = polyshape(D);

F = [ 1281 749;
1252 644;
1338 348;
1440 528;
1374 723];
fgon = polyshape(F);

G = [ 162 828;
195 1101;
308 1135;
295 790];
ggon = polyshape(G);

H = [ 434 869;
526 1139;
659 1199;
603 787];
hgon = polyshape(H);

I = [ 897 963;
818 1288
917 1318
1076 1187
1089 914];
igon = polyshape(I);

J = [1182 1206
1328 1273
1440 1030
1215 963];
jgon = polyshape(J);

K= [1464 880
1606 944
1745 738
1656 693];
kgon = polyshape(K);

L = [ 1619 1067
1695 1281
2007 1202
1884 966];
lgon = polyshape(L);

M = [ 119 1476
281 1790
394 1513
311 1202
179 1251];
mgon = polyshape(M);

N = [ 579 1783
626 1955
798 1884
877 1629];
ngon = polyshape(N);

O = [ 1089 1854
1146 1891
1377 1850
1318 1532
1182 1554];
ogon = polyshape(O);

Q = [ 1404 1663
1487 1768
1619 1479
1450 1322];
qgon = polyshape(Q);

plot(egon)
hold on
plot(tgon)
hold on
plot(pgon)
hold on
plot(dgon)
hold on
plot(fgon)
hold on
plot(ggon)
hold on
plot(hgon)
hold on
plot(igon)
hold on
plot(jgon)
hold on
plot(kgon)
hold on
plot(lgon)
hold on
plot(mgon)
hold on
plot(ngon)
hold on
plot(ogon)
```

```

hold on
plot(qgon)
axis([0 2000 0 2000])
grid on

% Plotting start position
circles(start(1), start(2),0.5, 'facecolor','green')

% Plotting goal position
circles(goal(1), goal(2),0.5, 'facecolor','yellow')

%% initialising the hash map

%% 1st Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j'};
% values = {start, [20,10], [20,40], [60,40],
[60,10], [50,60], [50,80], [70,80], [70,60], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 2nd Config
% keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h',
'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r'};
% values = {start, [20,10], [20,30], [60,30],
[60,10], [10,40], [10,60], [50,60], [50,40],
[70,10], [70,50], [90,50], [90,10], [20,70],
[20,90], [80,90], [80,70], goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%% 3rd Config
keys = {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i',
'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's',
't', 'u', 'v', 'w', 'x', 'y', 'z'...
'aa', 'ab', 'ac', 'ad', 'ae', 'af', 'ag', 'ah',
'ai', 'aj', 'ak', 'al', 'am', 'an', 'ao', 'ap',
'aq', 'ar', 'as', 'at', 'au', 'av', 'aw', 'ax',
'ay', 'az'...
'ba', 'bb', 'bc', 'bd', 'be', 'bf', 'bg', 'bh',
'bi', 'bj', 'bk', 'bl', 'bm', 'bn', 'bo'};
values = {start, [573 637],[699 644],[831
513],[659 371],[579 513]...
[374 210],[500 375],[864 22],[440
15]...
[30 584], [232 697],[368 577],[248 438]...
[162 828],[195 1101],[308 1135],[295 790]...
[881 667],[1063 685],[1182 506],[934 468]...
[1281 749],[1252 644],[1338 348],[1440 528],[1374
723]...
[434 869],[526 1139],[659 1199],[603 787]...
[897 963],[818 1288],[917 1318],[1076
1187],[1089 914]...
[1182 1206],[1328 1273],[1440 1030],[1215
963]...
[1464 880],[1606 944],[1745 738],[1656
693]...
[1619 1067],[1695 1281],[2007 1202],[1884
966]...
[119 1476],[281 1790],[394 1513],[311
1202],[179 1251]...
[579 1783],[626 1955],[798 1884],[877
1629]...
[1089 1854],[1146 1891],[1377 1850],[1318
1532],[1182 1554]...
[1404 1663],[1487 1768],[1619 1479],[1450
1322]...
goal};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
pp = length(values);

% for i = 1:pp
%     for j = 2:pp
%         temp = [values{1,i}(1), values{1,j}(1),
values{1,i}(2), values{1,i+1}(2)];
%         edges = vertcat(edges, temp);
%     end

```

```

% end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Map = containers.Map(keys, values);
len = Map.values;
Map('j');

%% Building all the obstacle edges
length = size(keys); % this contains the number
of nodes

edges = [];

for i = 2:(length(2)-2)
    temp = [values{1,i}(1), values{1,i+1}(1),
values{1,i}(2), values{1,i+1}(2)];
    edges = vertcat(edges, temp);
end

% Removing edges which are not obstacle edges
% sizeEdges = size(edges);
% i = 4;
% while i < (sizeEdges(1)-1)
%     [edges,ps] = removerows(edges,'ind',i);
%     sizeEdges = size(edges);
%     i = i + 3;
% end

% Adding 1 edge pair in each obstacle which were not
added in the earlier
% for loop
i = 2;
while i < length(2)-2
    temp = [values{1,i}(1), values{1,i+3}(1),
values{1,i}(2), values{1,i+3}(2)];
    edges = vertcat(edges, temp);
    i = i + 4;
end

%% Calculating the valid edges and adding them to
the graph
ledgeSize = size(edges);
noEdges = ledgeSize(1);

G = graph();

for i = 1:length(2)
    for j = (i + 1):length(2)
        % find equation of the edge to be checked
        p1 = values{1,i};
        q1 = values{1,j};
        m1 = (q1(2)-p1(2))/(q1(1)-p1(1)); %slope
        c1 = p1(2) - m1*(p1(1));
        %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

        flag = 1; % flag to check if the edge has
any intersection with any other edge;
        % '1' means no intersection
        % need to compare with the edges
        for k = 1:noEdges

            ed = edges(k,:);
            m2 = (ed(4) - ed(3))/(ed(2) - ed(1));
            if(ed(2)==ed(1))
                m2 = 1e+10;
            end
            c2 = ed(3) - m2*ed(1);

            if m1==m2 %% ignoring
                t = 1;
            else

                %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
                temp1 = ed(3) - m1*ed(1) - c1;
                temp2 = ed(4) - m1*ed(2) - c1;

```

```

        temp3 = p1(2) - m2*p1(1) - c2;
        temp4 = q1(2) - m2*q1(1) - c2;

        if (sign(temp1) ~= sign(temp2)) &&
sign(temp1)~=0 && sign(temp2)~=0 && (sign(temp3) ~=
sign(temp4)) && sign(temp3)~=0 && sign(temp4)~=0
            flag = 0;
            break
        end
        %%%%%
    end

end

if flag==1
    G = addedge(G,keys{i}, keys{j});
end

end

end

%% Removing the diagonals of the obstacle from the
visible edges
length = size(keys);    % this contains the number
of nodes
i = 2;
pair = [keys; values]; % this is used to find the
corresponding letter with its point and what number
it is on the list
while i < (length(2)-2)

    G = rmedge(G,keys{i}, keys{i+2});
    G = rmedge(G,keys{i+1}, keys{i+3});
    i = i + 4;
end
%
G = rmedge(G,keys{12}, keys{14});
G = rmedge(G,keys{11}, keys{16});
% G = rmedge(G,keys{17}, keys{8});
% G = rmedge(G,keys{20}, keys{15});
% G = rmedge(G,keys{2}, keys{13});
% % G = rmedge(G,keys{9}, keys{12});

%% Plotting all the visible edges

visEd = G.Edges; % this has the visible edges
sizeEd = size(G.Edges);

for i=1:sizeEd(1)
    x = visEd(i,1);
    xx = x{1,1};
    p1 = Map(xx{1,1});
    p2 = Map(xx{1,2});
    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];
    hold on
    plot(xpoints, ypoints, 'b')
end

%% finding the shortest path in the graph and
printing it

path = shortestpath(G, keys{1}, keys{length(2)});
pathSize = size(path);

totalDis = 0;    % this will contain the total
distance in units for the final selected path

for i=1:pathSize(2)-1
    p1 = Map(path{i});
    p2 = Map(path{i+1});

    totalDis = totalDis + EuclDist(p1,p2);

    xpoints = [p1(1,1), p2(1,1)];
    ypoints = [p1(1,2), p2(1,2)];

```

```

        hold on
        plot(xpoints, ypoints, 'k', 'LineWidth', 3)
        title('Visibility Graph')
    end

    pairm = [keys; values];

    toc
    totalDis
    %%%%%%%%% end
    %%%%%%%%%

```
